ASSURE DEFI®

THE VERIFICATION GOLD STANDARD

# SECURITY ASSESSMENT REPORT

VERIFIED BY ASSURE DEFI®

INTEGRITY • TRUST • CREDIBILITY

| | |
|---|---|
| NAME: | GG PROTOCOL |
| STATUS: | FAIL |
| DATE: | 16/04/2026 |

# Risk Analysis

## Vulnerability summary

| Classification | Description |
|---|---|
| High | Vulnerabilities that can lead to **loss or theft of funds**, **permanent locking of assets**, **unauthorized minting/burning**, **protocol insolvency**, or **full control of the contract or protocol**.<br>These issues are typically **exploitable on-chain** and require immediate remediation. |
| Medium | Vulnerabilities that **weaken protocol security or trust assumptions** but do not directly enable immediate loss of funds under normal conditions.<br>Exploitation may require **specific conditions, privileged roles, or external failures**. |
| Low | Issues that have **limited direct impact** on funds or protocol integrity but may cause **unexpected behavior**, **reduced resilience**, or **future exploitability** if combined with other weaknesses. |
| Informational | Findings that **do not represent security vulnerabilities**, but highlight **code quality, clarity, maintainability, or best-practice improvements** that may enhance long-term safety and auditability. |

# SCOPE

## Target Code And Revision

| | |
|---|---|
| **Project** | Assure |
| **Codebase** | https://github.com/GlobalGoldFinance/gg_protocol_contracts<br>Commit:<br>1c0ee093a11ad2ddc632aae584c66ddbe31be181 |
| **Audit Methodology** | Static, Manual |

# Detailed Technical Report

HIGH

**1. Vault operator can mint under-collateralized / inflated NFTs, then mint unbacked ERC20 against them**

**Location**:

contracts/tokens/nft/GGPreciousMetalUtility.sol:260-347

contracts/tokens/nft/GGGoldBar.sol:141-178

contracts/tokens/nft/GGSilverBar.sol:143-180

**Issue**:

tokenizeFromInventory() only commits to:recordId, vaultId, assetClassId, assetTypeId, pmCollectionId ,salt

It does not bind or validate:

grossWeightOzt, purity, brand ,mintYear, serialNumber

Then GGGoldBar.mintFromInventory() / GGSilverBar.mintFromInventory() directly trust those caller-supplied values and compute:

```
fineGoldOzt = grossWeightOzt * purity / 10000
fineSilverOzt = grossWeightOzt * purity / 10000
```

That value later becomes the collateral token amount via getCollateralData(), and _executeCollateralize() mints that amount of ERC20 to the owner.

The code documents the missing validation with a TODO:

GGGoldBar.sol:154-161

GGSilverBar.sol:156-163

**Remediation:**

Bind all economically relevant bar fields into the inventory commitment and/or validate them onchain against an authoritative registry.

At minimum:

include grossWeightOzt, purity, brandHash, mintYear, and ideally serialNumberHash in the committed record

enforce asset-type-specific validation in mintFromInventory()

reject tokenization unless metadata exactly matches the committed / approved

inventory data

## 2. Anyone can erase another users delinquency and bypass fee-based transfer restrictions

**Location**:

contracts/registries/GGStorageFee.sol:210-218

**Issue**:

updateStorageFee(address user_, uint256 amount_) is external and has no access control.

Any caller can do:

```
updateStorageFee(victim, hugeAmount)
```

and force:

delinquencies[victim] down to zero

totalFeesCollected artificially up

This is not just a bookkeeping issue. The delinquency state is consumed by enforcement paths:

```
ERC20 transfer diversion logic in GGPreciousMetalERC20._update()
NFT delinquency blocking in GGPreciousMetalUtility.checkDelinquency()
```

**Remediation:**

Restrict updateStorageFee() to a trusted caller set, such as:

the ERC20 contract that diverts payments

the utility contract

a designated fee collector / treasury module

Also:

emit an event

validate the payer/context

consider removing the duplicate action path if Guardian actions are the intended control surface

**3. Vault-specific delinquency is never enforced**

**Location**:

Setter exists: contracts/registries/GGStorageFee.sol:144-149

Getter exists: contracts/registries/GGStorageFee.sol:189-202

Enforcement ignores vault delinquency:

contracts/registries/GGStorageFee.sol:243-260

contracts/tokens/nft/GGPreciousMetalUtility.sol:204-221

**Issue**:

The contract supports a separate vaultDelinquencies[vaultId][user] mapping and a Guardian action SET_VAULT_DELINQUENCY, which strongly implies per-vault fee enforcement is intended.

But the actual enforcement functions:

```
isAccountDelinquent(account, vaultId)
isAccountsDelinquent(accounts, vaultId)
```

ignore vaultId entirely and only check the global delinquencies[user].

Then GGPreciousMetalUtility.checkDelinquency() passes a vaultId into isAccountsDelinquent(...), but that vaultId is discarded.

**Remediation:**

Use vault-specific data in enforcement:

isAccountDelinquent(account, vaultId) should check vaultDelinquencies[vaultId][account]

isAccountsDelinquent(accounts, vaultId) should do the same

if both global and vault delinquency are intended, enforce the union of both

Also update payment-settlement paths so reducing delinquency updates the correct scope.

**4. Redeemed precious metal NFTs remain transferable and potentially collateralizable**

**Location**:

/nft/GGPreciousMetalNFTBase.sol — redeem()

nft/GGPreciousMetalNFTBase.sol — _update()

/nft/GGPreciousMetalUtility.sol — collateralization path

**Issue**:

redeem(tokenId_) marks a token as redeemed by setting preciousMetalData[tokenId_].isRedeemed = true, but based on the reported implementation, the protocol does not consistently enforce redeemed-state exclusion across all downstream lifecycle paths. In particular, the transfer hook reportedly does not explicitly reject redeemed tokens, and the collateralization flow checks isCollateralized but not isRedeemed.

This creates a protocol-invariant gap: a redeemed NFT may remain in circulation or be processed by later actions unless all such paths independently rely on existence checks that treat isRedeemed as non-existent.

**Remediation:**

Harden redeemed-state handling everywhere:

revert on transfer of redeemed NFTs

revert on collateralization of redeemed NFTs

strongly consider burning redeemed NFTs, or permanently freezing them

ensure all read/write lifecycle functions treat redeemed tokens as terminally inactive

## 5. Unrestricted UUPS upgrades allow complete protocol takeover

**Location**:

GGPreciousMetalERC20.sol L591-596 _authorizeUpgrade performs no access control, so upgradeTo on the USG token proxy succeeds for any caller, enabling arbitrary logic that can mint or drain tokens.

GGPreciousMetalUtility.sol L820-822 The utility contract that handles collateralization also leaves _authorizeUpgrade empty, letting anyone upgrade it and bypass all vault/compliance checks.

GGVaultRegistry.sol L784-789 VaultRegistry's upgrade hook has no authorization, so its proxy can be replaced and vault mappings/inventory clones hijacked.

GGVaultInventoryRecord.sol L553-558 Inventory records rely on _authorizeUpgrade with no checks, allowing upgrades that can rewrite tokenization links and record state.

GGCompliance.sol L463-468 Compliance contract likewise lacks upgrade protection, letting an attacker upgrade to always-true checks and bypass transfer restrictions.

**Issue**:

Multiple upgradeable contracts implement _authorizeUpgrade without any access control, allowing any caller to upgrade proxies to malicious implementations and seize assets.

**Remediation:**

Enforce strict access control in _authorizeUpgrade, for example, by restricting to the Guardian/role-based governance contract or using the existing GuardianEnterprise action flow to set a pending implementation before upgrading. Consider disabling external upgrade functions or inheriting OpenZeppelin's UUPSUpgradeable pattern with proper authorization checks.

## 6. Missing approved-client check in collateralization allows unauthorized vault participation

**Location**:

contracts/tokens/nft/PreciousMetalUtility.sol

_executeCollateralize()

collateralization execution path

**Issue**:

The documentation explicitly states that collateralization should require the user to already be an approved client of the relevant vault. The same document then says this prerequisite is not enforced in _executeCollateralize().

If accurate, that means a wallet holding a valid NFT can collateralize it and mint backing ERC20 without being an approved vault client.

This is more than a workflow mismatch. In this protocol, vault-client approval appears tied to compliance, KYC, and vault-level permissioning. Failing to enforce it at the actual asset-conversion point lets unauthorized users enter the backed-token system.

**Remediation:**

In the collateralization execution path, require that the NFT owner is an approved client of the NFT's vault before minting ERC20.

At minimum:

resolve vaultId from the NFT

check VaultClientRegistry for owner approval at execution time

revert if the owner is not currently approved

**7. Direct NFT transfers remain possible outside the vault approval flow, breaking title-transfer controls**

**Location**:

contracts/tokens/nft/GGPreciousMetalNFTBase.sol

_update(...)

ERC721 transfer path

**Issue**:

The protocol docs describe NFT transfer as a controlled process requiring off-chain title handling and Guardian-mediated approval. But the divergence doc explicitly states that PreciousMetalNFTBase._update() still allows direct transfers of uncollateralized NFTs.

If true, then holders can transfer NFTs peer-to-peer onchain without the protocol executing the intended title-transfer workflow.

In an ordinary NFT collection that would be fine. In a real-world-asset protocol, it is a serious integrity problem because the onchain holder may diverge from the party with legal or vault-recognized title.

**Remediation:**

Block all direct transfers except those initiated by the designated utility/execution contract during an approved transfer request flow.

At minimum:

revert in _update() for user-initiated transfers

allow only mint, burn, or utility-authorized transfer execution

enforce recipient compliance and vault title settlement before final transfer

**8. Vault can revoke a client while the client still holds active vault-linked NFTs**

**Location**:

contracts/registries/GGVaultClientRegistry.sol

revokeClient(...)

**Issue**:

The vault client registry allows a vault operator to revoke a client even when that client still holds active NFTs associated with the vault.

This breaks a core state invariant between:

vault client registration status, and

ownership of active vault-linked assets.

If active ownership of vault assets requires valid client status, then revocation should be blocked until the user no longer holds unresolved assets.

**Remediation:**

Before revoking a client, enforce that the wallet has no unresolved vault-linked assets.

At minimum:

query all relevant NFT collections for active holdings

revert if the wallet still owns active vault-linked NFTs

alternatively freeze the assets first, or require transfer/redemption before revocation

**9. forceCollateralizeForFees() is broken because the utility pulls backing tokens from the NFT contract address**

**Location**:

contracts/tokens/nft/GGPreciousMetalUtility.sol

forceCollateralizeForFees(...)

_transferTokens(...)

**Issue**:

The forced-collateralization path intended for fee enforcement is nonfunctional because it attempts to transfer backing ERC20 tokens from the NFT contract address rather than the actual token holder or a valid reserve source.

As a result, forceCollateralizeForFees(...) can revert instead of collateralizing the NFT and applying fee collection.

**Remediation:**

Use the correct funding source for the ERC20 transfer.

At minimum:

determine whether the source should be the NFT owner, a reserve, or a protocol-controlled backing account

do not use the NFT contract address as the ERC20 source unless that is explicitly true and approved

add end-to-end tests proving forced collateralization succeeds and updates collateral state correctly

MEDIUM

## 1. findVaultNFTContractsForAssetType() ignores collection asset-type restrictions

**Location**:

contracts/registries/GGContractRegistry.sol

findVaultNFTContractsForAssetType(...)

_isAssetTypeAllowed(...)

**Issue**:

The registry lookup intended to return NFT contracts for a specific vault, asset class, and asset type does not actually enforce asset-type restrictions at the collection level.

Although findVaultNFTContractsForAssetType(...) appears to filter using _isAssetTypeAllowed(...), the helper currently returns true unconditionally. This means asset-type filtering is effectively disabled.

Code evidence

findVaultNFTContractsForAssetType(...) uses _isAssetTypeAllowed(collection.nftContract, assetTypeId_) as part of the filter.

But _isAssetTypeAllowed(...) currently contains logic equivalent to:

```
return true;
```

with a comment indicating this is temporary.

**Remediation:**

Implement real asset-type filtering.

At minimum:

query the NFT collection's supported/allowed asset type configuration

return false when the requested type is not permitted

add tests proving mixed collections in the same vault/class are filtered correctly by asset type

## LOW

No low issues were found.

## INFORMATIONAL

No informational issues were found.

# Testing coverage

During the testing phase, custom use cases were written to cover all the logic of contracts. *Check "Annexes" to see the testing code.*

```
  GG functional security suite - compilation
Nothing to compile
    ✓ extracts and compiles the GG source tree (379ms)
    ✓ contains the expected high-value GG modules

  GG functional security suite - static risk gates
    ✓ has no critical dangerous primitives
Inline assembly findings: 18
[
  {
    "file": "extracted/contracts/core/GGCompliance.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/core/GGTransferLogger.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGAssetTypeRegistry.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGContractRegistry.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGNFTBeaconFactory.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGProtocolFeeRegistry.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGStorageFee.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGVaultInventoryRecord.sol",
    "rule": "high-inline-assembly",
```

```
  },
  {
    "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGVaultInventoryRecord.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  },
  {
    "file": "extracted/contracts/registries/GGVaultRegistry.sol",
    "rule": "high-inline-assembly",
    "severity": "high",
    "match": "assembly {",
    "note": "Inline assembly present."
  }
]

  ▯ reports inline assembly occurrences for manual review only

GG functional security suite - attack surface
  ▯ discovers public and external functions
  ▯ finds sensitive administrative entrypoints

GG functional security suite - access-control heuristics
Sensitive production mutator candidates for manual review: 58
[
  {
    "file": "extracted/contracts/core/GGCompliance.sol",
    "function": "initialize"
  },
  {
    "file": "extracted/contracts/core/GGTransferLogger.sol",
    "function": "initialize"
  },
  {
    "file": "extracted/contracts/interfaces/IGGContractRegistry.sol",
    "function": "addVaultNFTContract"
  },
  {
    "file": "extracted/contracts/interfaces/IGGContractRegistry.sol",
    "function": "addVaultInventoryContract"
  },
  {
    "file": "extracted/contracts/interfaces/IGGContractRegistry.sol",
    "function": "setVaultPreciousMetalNFTContractActiveStatus"
  },
  {
    "file": "extracted/contracts/interfaces/IGGNFTBeaconFactory.sol",
    "function": "deployBeaconProxy"
  },
  {
    "file": "extracted/contracts/interfaces/IGGPreciousMetalERC20.sol",
    "function": "initialize"
  },
  {
    "file": "extracted/contracts/interfaces/IGGPreciousMetalERC20.sol",
    "function": "mintFromUtility"
  },
  {
    "file": "extracted/contracts/interfaces/IGGPreciousMetalERC20.sol",
    "function": "burnFromUtility"
  },
```

```
{
  "file": "extracted/contracts/interfaces/IGGPreciousMetalMintable.sol",
  "function": "mintFromInventory"
},
{
  "file": "extracted/contracts/interfaces/IGGPreciousMetalNFT.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/interfaces/IGGPreciousMetalNFT.sol",
  "function": "markCollateralized"
},
{
  "file": "extracted/contracts/interfaces/IGGStorageFee.sol",
  "function": "updateStorageFee"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "markTokenized"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "markUntokenized"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "setAuthorizedCaller"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "initializeClone"
},
{
  "file": "extracted/contracts/registries/GGAssetTypeRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "addVaultNFTContract"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "addVaultInventoryContract"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "setVaultPreciousMetalNFTContractActiveStatus"
},
{
  "file": "extracted/contracts/registries/GGNFTBeaconFactory.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGNFTBeaconFactory.sol",
  "function": "deployBeaconProxy"
},
{
  "file": "extracted/contracts/registries/GGProtocolFeeRegistry.sol",
  "function": "initialize"
},
```

```
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "registerClient"
},
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "registerClientBatch"
}

  reviews sensitive production mutators for access-control signals
ddress-sensitive production functions for zero-address review: 34

{
  "file": "extracted/contracts/core/GGCompliance.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/core/GGTransferLogger.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/interfaces/IGGContractRegistry.sol",
  "function": "addVaultNFTContract"
},
{
  "file": "extracted/contracts/interfaces/IGGContractRegistry.sol",
  "function": "addVaultInventoryContract"
},
{
  "file": "extracted/contracts/interfaces/IGGContractRegistry.sol",
  "function": "setVaultPreciousMetalNFTContractActiveStatus"
},
{
  "file": "extracted/contracts/interfaces/IGGPreciousMetalERC20.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/interfaces/IGGPreciousMetalNFT.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/interfaces/IGGStorageFee.sol",
  "function": "updateStorageFee"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "setAuthorizedCaller"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "initializeClone"
},
{
  "file": "extracted/contracts/registries/GGAssetTypeRegistry.sol",
  "function": "initialize"
```

```
  "file": "extracted/contracts/interfaces/IGGPreciousMetalERC20.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/interfaces/IGGPreciousMetalNFT.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/interfaces/IGGStorageFee.sol",
  "function": "updateStorageFee"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "setAuthorizedCaller"
},
{
  "file": "extracted/contracts/interfaces/IGGVaultInventoryRecord.sol",
  "function": "initializeClone"
},
{
  "file": "extracted/contracts/registries/GGAssetTypeRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "addVaultNFTContract"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "addVaultInventoryContract"
},
{
  "file": "extracted/contracts/registries/GGContractRegistry.sol",
  "function": "setVaultPreciousMetalNFTContractActiveStatus"
},
{
  "file": "extracted/contracts/registries/GGNFTBeaconFactory.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGProtocolFeeRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGStorageFee.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGStorageFee.sol",
  "function": "updateStorageFee"
},
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "registerClient"
},
```

```
{
  "file": "extracted/contracts/registries/GGProtocolFeeRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGStorageFee.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGStorageFee.sol",
  "function": "updateStorageFee"
},
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "registerClient"
},
{
  "file": "extracted/contracts/registries/GGVaultClientRegistry.sol",
  "function": "registerClientBatch"
},
{
  "file": "extracted/contracts/registries/GGVaultInventoryRecord.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGVaultInventoryRecord.sol",
  "function": "initializeClone"
},
{
  "file": "extracted/contracts/registries/GGVaultInventoryRecord.sol",
  "function": "setAuthorizedCaller"
},
{
  "file": "extracted/contracts/registries/GGVaultRegistry.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/registries/GGVaultRegistry.sol",
  "function": "updateVaultTreasury"
},
{
  "file": "extracted/contracts/tokens/erc20/GGAllocationManager.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/tokens/erc20/GGPreciousMetalERC20.sol",
  "function": "initialize"
},
{
  "file": "extracted/contracts/tokens/nft/GGGoldBar.sol",
  "function": "initialize"
}
]
```


reviews production address-setting functions for zero-address validation manually

```
  GG functional security suite - control plane coverage
    ✓ contains pause, role, and compliance control plane modules
    ✓ exposes emergency, role, or compliance logic in source text

  Runtime security - property-based tests
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ ERC20 preserves total supply and balance conservation across randomized transfers (1094ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ NFT minting maintains unique ownership and sequential token existence (172ms)

  Runtime security - fuzzing tests
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ fuzzes compliance-gated ERC20 transfers across randomized allow/deny patterns (503ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ fuzzes ERC20 delinquency routing to treasury (338ms)

  Runtime security - invariant testing
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ maintains ERC20 ledger invariants under mixed randomized operations (6870ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ maintains NFT ownership invariants during transfers and collateralization toggles

  Runtime security - exploit reproduction and auth abuse
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ reproduces and blocks unauthorized mint/burn/pause attempts on ERC20 (112ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ reproduces and blocks direct transfer of collateralized NFT outside utility path (107ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ reproduces and blocks unauthorized NFT emergency pause toggles (96ms)

  Runtime security - reentrancy and runtime abuse simulations
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ blocks ERC20 reentrancy via malicious utility/logging callback (99ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegis
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_A
    ✓ blocks NFT reentrancy via malicious utility callback during transfer logging (100ms)


  21 passing (3s)
```

```
  Runtime security - property-based tests:runtime
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ ERC20 preserves total supply and balance conservation across randomized transfers (2039ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ NFT minting maintains unique ownership and sequential token existence (188ms)

  Runtime security - fuzzing tests
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ fuzzes compliance-gated ERC20 transfers across randomized allow/deny patterns
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ fuzzes ERC20 delinquency routing to treasury (355ms)

  Runtime security - invariant testing
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ maintains ERC20 ledger invariants under mixed randomized operations (699ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ maintains NFT ownership invariants during transfers and collateralization toggles (160ms)

  Runtime security - exploit reproduction and auth abuse
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ reproduces and blocks unauthorized mint/burn/pause attempts on ERC20 (107ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ reproduces and blocks direct transfer of collateralized NFT outside utility path (99ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ reproduces and blocks unauthorized NFT emergency pause toggles (93ms)

  Runtime security - reentrancy and runtime abuse simulations
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ blocks ERC20 reentrancy via malicious utility/logging callback (102ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"cont
ter","name":"guardianRouter_","type":"IGuardianRouter"}],"name":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.16.0)
    ✓ blocks NFT reentrancy via malicious utility callback during transfer logging (114ms)


  11 passing (4s)
```

```
  Runtime security - property-based tests
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGuardianRouter","name":"guardianRouter_","type":"IGuardianRouter"}],"name
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.1
6.0)
    ✓ ERC20 preserves total supply and balance conservation across randomized transfers (1670ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes","name":"data_","type":"bytes"},{"internalType":
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGuardianRouter","name":"guardianRouter_","type":"IGuardianRouter"}],"name
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="IRoleRegistry", code=INVALID_ARGUMENT, version=6.1
6.0)
    ✓ NFT minting maintains unique ownership and sequential token existence (182ms)


  2 passing (2s)
```

```
  Runtime security - fuzzing tests
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGuardianRouter","name":"
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="
6.0)
    ✓ fuzzes compliance-gated ERC20 transfers across randomized allow/deny patterns (1168ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"},{"internalType":"bytes
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract IGuardianRouter","name":"
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (argument="type", value="
6.0)
    ✓ fuzzes ERC20 delinquency routing to treasury (343ms)


  2 passing (2s)
```

```
  Runtime security - invariant testing
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract I
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (a
6.0)
    ✓ maintains ERC20 ledger invariants under mixed randomized operations (824ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes32"
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contract I
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type (a
6.0)
    ✓ maintains NFT ownership invariants during transfers and collateralization toggles (166ms)


  2 passing (993ms)
```

```
  Runtime security - exploit reproduction and auth abuse
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contrac
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type
6.0)
    ✓ reproduces and blocks unauthorized mint/burn/pause attempts on ERC20 (691ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contrac
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type
6.0)
    ✓ reproduces and blocks direct transfer of collateralized NFT outside utility path (111ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"bytes
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"contrac
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid type
6.0)
    ✓ reproduces and blocks unauthorized NFT emergency pause toggles (98ms)


  3 passing (7s)
```

```
  Runtime security - reentrancy and runtime abuse simulations
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"b
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"con
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid
5.0)
    ✓ blocks ERC20 reentrancy via malicious utility/logging callback (693ms)
[Warning] Invalid Fragment {"inputs":[{"internalType":"bytes32","name":"actionId_","type":"b
"contract IRoleRegistry","name":"roleRegistry_","type":"IRoleRegistry"},{"internalType":"con
":"validateBuiltInAction","outputs":[],"stateMutability":"view","type":"function"}: invalid
5.0)
    ✓ blocks NFT reentrancy via malicious utility callback during transfer logging (119ms)


  2 passing (815ms)
```

# Annexes

Testing code:

Property based runtime:

```
const { expect } = require('chai');
const { deployRuntimeFixture, randBetween } = require('./runtime/helpers');

describe('Runtime security - property-based tests', function () {
  it('ERC20 preserves total supply and balance conservation across randomized transfers',
async function () {
    const { token, alice, bob, charlie } = await deployRuntimeFixture();
    const users = [alice, bob, charlie];
    const initialMints = [1000n, 2000n, 3000n];

    for (let i = 0; i < users.length; i++) {
      await (await token.testMint(users[i].address, initialMints[i])).wait();
    }

    const initialSupply = await token.totalSupply();

    for (let i = 0; i < 50; i++) {
      const fromIdx = randBetween(`from-${i}`, 0, 2);
      let toIdx = randBetween(`to-${i}`, 0, 2);
```

```
      if (toIdx === fromIdx) toIdx = (toIdx + 1) % 3;

      const from = users[fromIdx];
      const to = users[toIdx];
      const balance = await token.balanceOf(from.address);
      const amount = balance === 0n ? 0n : BigInt(randBetween(`amt-${i}`, 0, Number(balance
> 50n ? 50n : balance)));

      await (await token.connect(from).transfer(to.address, amount)).wait();

      const totalSupply = await token.totalSupply();
      const aggregateBalances =
        await token.balanceOf(alice.address) +
        await token.balanceOf(bob.address) +
        await token.balanceOf(charlie.address);

      expect(totalSupply).to.equal(initialSupply);
      expect(aggregateBalances).to.equal(initialSupply);
    }
  });

  it('NFT minting maintains unique ownership and sequential token existence', async
function () {
    const { nft, alice, bob, charlie } = await deployRuntimeFixture();
    const users = [alice, bob, charlie];
    const mintedIds = [];

    for (let i = 0; i < 20; i++) {
      const recipient = users[randBetween(`recipient-${i}`, 0, 2)];
      const tx = await nft.testMintDirect(recipient.address, i + 1);
      await tx.wait();
      mintedIds.push(BigInt(i + 1));
    }

    const seen = new Set();
    for (const tokenId of mintedIds) {
      const owner = await nft.ownerOf(tokenId);
      expect(owner).to.not.equal('0x0000000000000000000000000000000000000000');
      expect(seen.has(tokenId.toString())).to.equal(false);
      seen.add(tokenId.toString());
    }

    expect(await nft.balanceOf(alice.address) + await nft.balanceOf(bob.address) + await
nft.balanceOf(charlie.address)).to.equal(20n);
  });
```

```
});
```

Fuzzing runtime:

```
const { expect } = require('chai');
const { expectRevert, expectSuccess } = require('./runtime/assertions');
const { deployRuntimeFixture, randBetween } = require('./runtime/helpers');

describe('Runtime security - fuzzing tests', function () {
  it('fuzzes compliance-gated ERC20 transfers across randomized allow/deny patterns', async
function () {
    const { token, compliance, alice, bob, charlie } = await deployRuntimeFixture();
    await (await token.testMint(alice.address, 5000n)).wait();

    const actors = [alice, bob, charlie];
    for (let i = 0; i < 40; i++) {
      const sender = actors[randBetween(`sender-${i}`, 0, 2)];
      const receiver = actors[randBetween(`receiver-${i}`, 0, 2)];
      await (await compliance.setUseAddressSpecificChecks(true)).wait();
      await (await compliance.setAddressCompliance(sender.address, randBetween(`s-ok-${i}`,
0, 1) === 1)).wait();
      await (await compliance.setAddressCompliance(receiver.address,
randBetween(`r-ok-${i}`, 0, 1) === 1)).wait();

      if (sender.address !== alice.address && (await token.balanceOf(sender.address)) ===
0n) {
        await (await compliance.setUseAddressSpecificChecks(false)).wait();
        await (await compliance.setAllowAll(true)).wait();
        await (await token.connect(alice).transfer(sender.address, 20n)).wait();
        await (await compliance.setUseAddressSpecificChecks(true)).wait();
      }

      const senderApproved = await compliance.isApprovedForUSGTransfer(sender.address);
      const receiverApproved = await compliance.isApprovedForUSGTransfer(receiver.address);
      const amount = 1n;

      if (senderApproved && receiverApproved) {
        await expectSuccess(token.connect(sender).transfer(receiver.address, amount));
      } else {
        await expectRevert(token.connect(sender).transfer(receiver.address, amount));
      }
    }
```

```
  });

  it('fuzzes ERC20 delinquency routing to treasury', async function () {
    const { token, storageFee, treasury, alice, bob } = await deployRuntimeFixture();
    await (await token.testMint(alice.address, 1000n)).wait();

    for (let i = 1; i <= 25; i++) {
      const delinquency = BigInt(randBetween(`delinq-${i}`, 0, 20));
      const transferAmount = BigInt(randBetween(`xfer-${i}`, 1, 25));
      await (await storageFee.setDelinquency(alice.address, delinquency)).wait();

      const aliceBefore = await token.balanceOf(alice.address);
      const bobBefore = await token.balanceOf(bob.address);
      const treasuryBefore = await token.balanceOf(treasury.address);

      await (await token.connect(alice).transfer(bob.address, transferAmount)).wait();

      const routedToTreasury = transferAmount <= delinquency ? transferAmount :
delinquency;
      const routedToBob = transferAmount <= delinquency ? 0n : transferAmount -
delinquency;

      expect(await token.balanceOf(alice.address)).to.equal(aliceBefore - transferAmount);
      expect(await token.balanceOf(treasury.address)).to.equal(treasuryBefore +
routedToTreasury);
      expect(await token.balanceOf(bob.address)).to.equal(bobBefore + routedToBob);
    }
  });
});
```

Invariant runtime:

```
const { expect } = require('chai');
const { deployRuntimeFixture, randBetween } = require('./runtime/helpers');

describe('Runtime security - invariant testing', function () {
  it('maintains ERC20 ledger invariants under mixed randomized operations', async function
() {
    const { token, alice, bob, charlie, treasury, storageFee } = await
deployRuntimeFixture();
    await (await token.testMint(alice.address, 3000n)).wait();
    await (await token.testMint(bob.address, 2000n)).wait();

    const tracked = [alice, bob, charlie, treasury];
    const initialSupply = await token.totalSupply();

    for (let step = 0; step < 60; step++) {
      const from = tracked[randBetween(`from-${step}`, 0, 2)];
      const to = tracked[randBetween(`to-${step}`, 0, 2)];
      const balance = await token.balanceOf(from.address);
      const amount = balance === 0n ? 0n : BigInt(randBetween(`amount-${step}`, 0,
Number(balance > 30n ? 30n : balance)));
      const delinquency = BigInt(randBetween(`d-${step}`, 0, 10));
      await (await storageFee.setDelinquency(from.address, delinquency)).wait();
      await (await token.connect(from).transfer(to.address, amount)).wait();

      const balances = await Promise.all(tracked.map((u) => token.balanceOf(u.address)));
      const sum = balances.reduce((acc, n) => acc + n, 0n);
      expect(sum).to.equal(initialSupply);
      expect(await token.totalSupply()).to.equal(initialSupply);
      for (const b of balances) expect(b >= 0n).to.equal(true);
    }
  });

  it('maintains NFT ownership invariants during transfers and collateralization toggles',
async function () {
    const { nft, alice, bob, charlie, registry } = await deployRuntimeFixture();
    await (await registry.setTrustSPV(charlie.address)).wait();
```

```
    for (let i = 1; i <= 10; i++) {
      await (await nft.testMintDirect(alice.address, i)).wait();
    }

    for (let tokenId = 1; tokenId <= 10; tokenId++) {
      if (tokenId % 2 === 0) {
        await (await nft.testSetCollateralized(tokenId, true)).wait();
        const data = await nft.preciousMetalData(tokenId);
        expect(data.isCollateralized).to.equal(true);
        expect(data.titleHolder).to.equal(charlie.address);
      } else {
        await (await nft.connect(alice).transferFrom(alice.address, bob.address,
tokenId)).wait();
        expect(await nft.ownerOf(tokenId)).to.equal(bob.address);
      }
    }

    expect(await nft.balanceOf(alice.address) + await
nft.balanceOf(bob.address)).to.equal(10n);
  });
});
```

Exploit repro:

```
const { expect } = require('chai');
const { expectRevert, expectSuccess } = require('./runtime/assertions');
const { deployRuntimeFixture } = require('./runtime/helpers');

describe('Runtime security - exploit reproduction and auth abuse', function () {
  it('reproduces and blocks unauthorized mint/burn/pause attempts on ERC20', async function
() {
    const { token, attacker, alice, pauseController } = await deployRuntimeFixture();

    await expectRevert(token.connect(attacker).mintFromUtility(attacker.address, 1n));
    await expectRevert(token.connect(attacker).burnFromUtility(attacker.address, 1n));
    await expectRevert(token.connect(attacker).emergencyPause());

    await (await token.testMint(alice.address, 10n)).wait();
    await (await token.connect(pauseController).emergencyPause()).wait();
    await expectRevert(token.connect(alice).transfer(attacker.address, 1n));
  });

  it('reproduces and blocks direct transfer of collateralized NFT outside utility path',
async function () {
```

```
    const { nft, alice, bob, charlie, registry } = await deployRuntimeFixture();
    await (await registry.setTrustSPV(charlie.address)).wait();
    await (await nft.testMintDirect(alice.address, 1)).wait();
    await (await nft.testSetCollateralized(1, true)).wait();

    await expectRevert(nft.connect(alice).transferFrom(alice.address, bob.address, 1));
  });

  it('reproduces and blocks unauthorized NFT emergency pause toggles', async function () {
    const { nft, attacker, pauseController } = await deployRuntimeFixture();
    await expectRevert(nft.connect(attacker).emergencyPause(true));
    await expectSuccess(nft.connect(pauseController).emergencyPause(true));
  });
});
```

Reentrancy abuse:

```
const { expect } = require('chai');
const { ethers } = require('hardhat');
const { expectRevert, expectSuccess } = require('./runtime/assertions');
const { deployRuntimeFixture } = require('./runtime/helpers');

describe('Runtime security - reentrancy and runtime abuse simulations', function () {
  it('blocks ERC20 reentrancy via malicious utility/logging callback', async function () {
    const { token, utility, alice, bob } = await deployRuntimeFixture();
    await (await token.testMint(alice.address, 100n)).wait();
    await (await utility.configureReentrancy(true, await token.getAddress(),
ethers.ZeroAddress, bob.address, 1, 0)).wait();

    await expectRevert(token.connect(alice).transfer(bob.address, 10n));
    expect(await token.balanceOf(alice.address)).to.equal(100n);
    expect(await token.balanceOf(bob.address)).to.equal(0n);
  });

  it('blocks NFT reentrancy via malicious utility callback during transfer logging', async
function () {
    const { nft, utility, alice, bob } = await deployRuntimeFixture();
    await (await nft.testMintDirect(alice.address, 1)).wait();
    await (await utility.configureReentrancy(true, ethers.ZeroAddress, await
nft.getAddress(), bob.address, 0, 1)).wait();

    await expectRevert(nft.connect(alice).transferFrom(alice.address, bob.address, 1));
    expect(await nft.ownerOf(1)).to.equal(alice.address);
  });
```

```
});
```

Foundry:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

import {GuardianEnterprise} from "../enterprise/GuardianEnterprise.sol";
import {GuardianRouter} from "../enterprise/GuardianRouter.sol";
import {RoleRegistry} from "../enterprise/RoleRegistry.sol";
import {Token} from "../examples/Token.sol";
import {
    CannotApproveOwnRequest,
    InvalidRequestOrder,
    NewRegistryNotConfigured,
    NotAuthorizedApprover
} from "../enterprise/utils/GuardianEnterpriseErrors.sol";

interface Vm {
    function expectRevert(bytes4 revertData) external;
    function expectRevert(bytes calldata revertData) external;
    function prank(address msgSender) external;
}

contract ConfirmedRoleRegistry is RoleRegistry {}

contract ConfirmedRouter is GuardianRouter {
    function bootstrapInitializeInRegistry(address admin_) external {
        roleRegistry.initializeContract(admin_);
    }

    function bootstrapAddAdmin(address member_) external {
        roleRegistry.addMember(roleRegistry.roleAdmin(), member_);
    }
}

contract ConfirmedEnterprise is GuardianEnterprise {
    constructor(address roleRegistry_, address guardianRouter_)
GuardianEnterprise(roleRegistry_, guardianRouter_) {}
```

```
    function bootstrapInitializeInRegistry(address admin_) external {
        roleRegistry.initializeContract(admin_);
    }

    function bootstrapCreateRole(bytes32 roleId_) external {
        roleRegistry.createRole(roleId_);
    }

    function bootstrapAddMember(bytes32 roleId_, address member_) external {
        roleRegistry.addMember(roleId_, member_);
    }
}

contract DummyGovernanceTarget {}

contract ConfirmedVulnerabilitiesTest {
    Vm internal constant vm = Vm(address(uint160(uint256(keccak256("hevm cheat
code")))));

    address internal constant ADMIN1 = address(0xA11CE);
    address internal constant ADMIN2 = address(0xB0B);
    address internal constant ATTACKER = address(0xBAD);
    address internal constant MEMBER = address(0xBEEF);
    address internal constant OWNER = address(0xABCD);
    address internal constant GUARDIAN1 = address(0x1111);
    address internal constant GUARDIAN2 = address(0x2222);
    address internal constant RECIPIENT = address(0xCAFE);

    bytes32 internal constant OPS_ROLE = keccak256("ROLE.OPS");
    bytes32 internal constant ROLE_ONE = keccak256("ROLE.ONE");
    bytes32 internal constant ROLE_TWO = keccak256("ROLE.TWO");

    /// @dev Reproduces the bootstrap deadlock where the only initial admin can submit
    ///      governance actions but can never collect the first approval because the
    ///      router forbids self-approval.
    function
test_EnterpriseBootstrap_WithSingleAdmin_CannotApproveFirstGovernanceRequest() public
{
        (ConfirmedRoleRegistry registry, ConfirmedRouter router) = _deployRouter();
        GuardianEnterprise target = new GuardianEnterprise(address(registry),
address(router));
        bytes32 createRoleAction = target.ACTION_CREATE_ROLE();

        target.initializeInRegistry(ADMIN1);

        vm.prank(ADMIN1);
        uint256 requestId = router.submitRequest(address(target), createRoleAction,
abi.encode(OPS_ROLE));
```

```
        vm.prank(ADMIN1);
        vm.expectRevert(CannotApproveOwnRequest.selector);
        router.approveRequest(address(target), requestId);

        vm.prank(ATTACKER);
        vm.expectRevert(NotAuthorizedApprover.selector);
        router.approveRequest(address(target), requestId);

        (, , , bool executed, , uint256 approvalCount, ) =
router.getPendingRequest(address(target), requestId);
        _assertFalse(executed, "bootstrap request should remain pending");
        _assertEq(approvalCount, 0, "single bootstrap admin should not be able to
approve");
    }

    /// @dev Reproduces the router self-governance queue bug: the router's own
    ///      `nextExecutableRequestId` is left at zero, so approved self-requests
    ///      can never satisfy the in-order execution check.
    function test_RouterSelfGovernance_QueuePointerStartsAtZero_AndBlocksExecution()
public {
        (, ConfirmedRouter router) = _deployRouter();
        bytes32 setTimeoutAction = router.ACTION_SET_REQUEST_TIMEOUT();

        router.bootstrapInitializeInRegistry(ADMIN1);
        router.bootstrapAddAdmin(ADMIN2);

        vm.prank(ADMIN1);
        uint256 requestId = router.submitRequest(address(router), setTimeoutAction,
abi.encode(uint256(2 hours)));

        vm.prank(ADMIN2);
        router.approveRequest(address(router), requestId);

        (, , , bool executed, , uint256 approvalCount, ) =
router.getPendingRequest(address(router), requestId);
        _assertFalse(executed, "router self-governance request should still be
pending");
        _assertEq(approvalCount, 1, "router request should have exactly one
approval");

        vm.expectRevert(abi.encodeWithSelector(InvalidRequestOrder.selector,
requestId));
        router.executeRequest(address(router), requestId);

        _assertEq(
            router.getRequestTimeout(address(router)),
            router.DEFAULT_TIMEOUT(),
            "self-governance should not be able to update the timeout"
```

```
        );
    }

    /// @dev Reproduces the zero-threshold configuration bug: assigning a custom
    ///      submitter role marks the action as custom-configured without setting a
    ///      replacement threshold, so the action becomes approval-less.
    function
test_CustomRoleConfiguration_ImplicitlySetsThresholdZero_AllowingApprovallessRouterUpd
ate() public {
        (ConfirmedRoleRegistry registry, ConfirmedRouter router) = _deployRouter();
        ConfirmedEnterprise target = new ConfirmedEnterprise(address(registry),
address(router));
        DummyGovernanceTarget maliciousRouter = new DummyGovernanceTarget();

        bytes32 configAction = target.ACTION_SET_SUBMITTER_ROLES();
        bytes32 updateRouterAction = target.ACTION_UPDATE_ROUTER();

        target.bootstrapInitializeInRegistry(ADMIN1);
        target.bootstrapAddMember(registry.roleAdmin(), ADMIN2);
        target.bootstrapCreateRole(OPS_ROLE);
        target.bootstrapAddMember(OPS_ROLE, ATTACKER);

        vm.prank(ADMIN1);
        uint256 configRequestId = router.submitRequest(
            address(target),
            configAction,
            abi.encode(updateRouterAction, _bytes32Array1(OPS_ROLE))
        );

        vm.prank(ADMIN2);
        router.approveRequest(address(target), configRequestId);

        _assertEq(
            registry.getThreshold(address(target), updateRouterAction),
            0,
            "custom role configuration should leave the action with threshold zero"
        );

        vm.prank(ATTACKER);
        uint256 exploitRequestId = router.submitRequest(
            address(target),
            updateRouterAction,
            abi.encode(address(maliciousRouter))
        );

        _assertEq(
            address(target.guardianRouter()),
            address(maliciousRouter),
            "attacker should be able to replace the router without approvals"
```

```
        );

        (, , , bool executed, , uint256 approvalCount, ) =
router.getPendingRequest(address(target), exploitRequestId);
        _assertTrue(executed, "exploit request should execute immediately");
        _assertEq(approvalCount, 0, "exploit request should succeed with zero
approvals");
    }

    /// @dev Reproduces the non-enterprise guardian initialization bug where the
    ///      default threshold remains zero, causing protected actions to
auto-execute.
    function
test_GuardianInitializer_LeavesDefaultThresholdZero_AndAutoExecutesProtectedAction()
public {
        Token token = new Token("Guardian Token", "GTK", OWNER,
_addressArray2(GUARDIAN1, GUARDIAN2));
        bytes32 mintAction = token.MINT_TOKENS();

        vm.prank(OWNER);
        token.submitActionRequest(
            mintAction,
            _uint256Array1(100),
            _addressArray1(RECIPIENT),
            new bytes32[](0)
        );

        _assertEq(token.balanceOf(RECIPIENT), 100, "protected mint should have
executed immediately");

        (, , , uint8 approvalCount, , bool finalized, , , ) = token.getRequest(1);
        _assertEq(uint256(approvalCount), 0, "mint request should not collect guardian
approvals");
        _assertTrue(finalized, "mint request should already be finalized");
    }

    /// @dev Reproduces the approver double-counting issue where the registry counts
    ///      the same address once per role, allowing governance to configure an
    ///      approval threshold that no set of unique voters can satisfy.
    function
test_OverlappingApproverRoles_AreDoubleCounted_CreatingUnreachableThreshold() public {
        (ConfirmedRoleRegistry registry, ConfirmedRouter router) = _deployRouter();
        ConfirmedEnterprise target = new ConfirmedEnterprise(address(registry),
address(router));
        DummyGovernanceTarget newRouter = new DummyGovernanceTarget();

        bytes32 roleAdmin = registry.roleAdmin();
        bytes32 setApproverRolesAction = target.ACTION_SET_APPROVER_ROLES();
        bytes32 setThresholdAction = target.ACTION_SET_ACTION_THRESHOLD();
```

```
        bytes32 updateRouterAction = target.ACTION_UPDATE_ROUTER();

        target.bootstrapInitializeInRegistry(ADMIN1);
        target.bootstrapAddMember(roleAdmin, ADMIN2);
        target.bootstrapCreateRole(ROLE_ONE);
        target.bootstrapCreateRole(ROLE_TWO);
        target.bootstrapAddMember(ROLE_ONE, MEMBER);
        target.bootstrapAddMember(ROLE_TWO, MEMBER);

        vm.prank(ADMIN1);
        uint256 setRolesRequestId = router.submitRequest(
            address(target),
            setApproverRolesAction,
            abi.encode(updateRouterAction, _bytes32Array2(ROLE_ONE, ROLE_TWO))
        );

        vm.prank(ADMIN2);
        router.approveRequest(address(target), setRolesRequestId);

        vm.prank(ADMIN1);
        uint256 setThresholdRequestId = router.submitRequest(
            address(target),
            setThresholdAction,
            abi.encode(updateRouterAction, uint8(2))
        );

        vm.prank(ADMIN2);
        router.approveRequest(address(target), setThresholdRequestId);

        _assertEq(
            registry.getApproverCount(address(target), updateRouterAction),
            2,
            "registry should incorrectly double-count the same approver"
        );

        vm.prank(ADMIN1);
        uint256 requestId = router.submitRequest(address(target), updateRouterAction,
abi.encode(address(newRouter)));

        vm.prank(ADMIN2);
        vm.expectRevert(NotAuthorizedApprover.selector);
        router.approveRequest(address(target), requestId);

        vm.prank(MEMBER);
        router.approveRequest(address(target), requestId);

        (, , , bool executed, , uint256 approvalCount, ) =
router.getPendingRequest(address(target), requestId);
        _assertFalse(executed, "request should remain stuck below the unreachable
```

```
threshold");
        _assertEq(approvalCount, 1, "only one unique approver should exist on-chain");
    }

    /// @dev Reproduces the registry migration deadlock: migration requires the new
    ///      registry to already contain this governed contract's admin state, but the
    ///      shipped wrapper only exposes initialization against the current registry.
    function
test_RegistryMigration_RequiresPreconfiguredTargetRegistry_ThatWrappersCannotSeed()
public {
        (ConfirmedRoleRegistry currentRegistry, ConfirmedRouter router) =
_deployRouter();
        ConfirmedRoleRegistry nextRegistry = new ConfirmedRoleRegistry();
        ConfirmedEnterprise target = new ConfirmedEnterprise(address(currentRegistry),
address(router));

        bytes32 roleAdmin = currentRegistry.roleAdmin();
        bytes32 migrateRegistryAction = target.ACTION_MIGRATE_REGISTRY();

        target.bootstrapInitializeInRegistry(ADMIN1);
        target.bootstrapAddMember(roleAdmin, ADMIN2);

        _assertEq(
            nextRegistry.roleMemberCount(address(target), nextRegistry.roleAdmin()),
            0,
            "replacement registry should start uninitialized for the governed
contract"
        );

        vm.prank(ADMIN1);
        uint256 requestId = router.submitRequest(address(target),
migrateRegistryAction, abi.encode(address(nextRegistry)));

        vm.prank(ADMIN2);
        vm.expectRevert(NewRegistryNotConfigured.selector);
        router.approveRequest(address(target), requestId);

        _assertEq(
            address(target.roleRegistry()),
            address(currentRegistry),
            "failed migration should leave the current registry unchanged"
        );
    }

    /// @dev Reproduces the threshold-decrement bug on guardian removal: once the
    ///      threshold is reduced from one to zero, future owner actions stop
requiring
    ///      any guardian approval.
    function
```

```
test_RemovingGuardian_DecrementsThresholdToZero_AndDisablesFutureGuardianApprovals()
public {
        Token token = new Token("Guardian Token", "GTK", OWNER,
_addressArray2(GUARDIAN1, GUARDIAN2));
        bytes32 setDefaultThresholdAction = token.GUARDIAN_SET_DEFAULT_THRESHOLD();
        bytes32 removeGuardianAction = token.GUARDIAN_REMOVE();
        bytes32 mintAction = token.MINT_TOKENS();

        vm.prank(OWNER);
        token.submitActionRequest(
            setDefaultThresholdAction,
            _uint256Array1(1),
            new address[](0),
            new bytes32[](0)
        );

        _assertEq(token.defaultGuardianThreshold(), 1, "threshold should first be
raised to one");

        vm.prank(OWNER);
        token.submitActionRequest(
            removeGuardianAction,
            new uint256[](0),
            _addressArray1(GUARDIAN2),
            new bytes32[](0)
        );

        vm.prank(GUARDIAN1);
        token.submitApproval(2);

        _assertEq(
            token.defaultGuardianThreshold(),
            0,
            "removing a guardian should not silently disable the guardian quorum"
        );

        vm.prank(OWNER);
        token.submitActionRequest(
            mintAction,
            _uint256Array1(100),
            _addressArray1(RECIPIENT),
            new bytes32[](0)
        );

        _assertEq(token.balanceOf(RECIPIENT), 100, "mint should auto-execute after the
threshold drops to zero");

        (, , , uint8 approvalCount, , bool finalized, , , ) = token.getRequest(3);
        _assertEq(uint256(approvalCount), 0, "mint request should not require guardian
```

```
approvals anymore");
        _assertTrue(finalized, "mint request should finalize immediately");
    }

    function _deployRouter() internal returns (ConfirmedRoleRegistry registry_,
ConfirmedRouter router_) {
        registry_ = new ConfirmedRoleRegistry();

        ConfirmedRouter implementation = new ConfirmedRouter();
        ERC1967Proxy proxy = new ERC1967Proxy(
            address(implementation),
            abi.encodeWithSelector(bytes4(keccak256("initialize(address)")),
address(registry_))
        );

        router_ = ConfirmedRouter(address(proxy));
    }

    function _addressArray1(address a_) internal pure returns (address[] memory arr_)
{
        arr_ = new address[](1);
        arr_[0] = a_;
    }

    function _addressArray2(address a_, address b_) internal pure returns (address[]
memory arr_) {
        arr_ = new address[](2);
        arr_[0] = a_;
        arr_[1] = b_;
    }

    function _bytes32Array1(bytes32 a_) internal pure returns (bytes32[] memory arr_)
{
        arr_ = new bytes32[](1);
        arr_[0] = a_;
    }

    function _bytes32Array2(bytes32 a_, bytes32 b_) internal pure returns (bytes32[]
memory arr_) {
        arr_ = new bytes32[](2);
        arr_[0] = a_;
        arr_[1] = b_;
    }

    function _uint256Array1(uint256 a_) internal pure returns (uint256[] memory arr_)
{
        arr_ = new uint256[](1);
        arr_[0] = a_;
    }
```

```
    function _assertEq(address actual_, address expected_, string memory message_)
internal pure {
        require(actual_ == expected_, message_);
    }

    function _assertEq(uint256 actual_, uint256 expected_, string memory message_)
internal pure {
        require(actual_ == expected_, message_);
    }

    function _assertFalse(bool condition_, string memory message_) internal pure {
        require(!condition_, message_);
    }

    function _assertTrue(bool condition_, string memory message_) internal pure {
        require(condition_, message_);
    }
}
```

# Technical Findings Summary

## Findings

| Vulnerability Level | Total | Pending | Not Apply | Acknowledged | Partially Fixed | Fixed |
|---|---|---|---|---|---|---|
| High | 9 | | | | | |
| Medium | 1 | | | | | |
| Low | | | | | | |
| Informational | | | | | | |

# Assessment Results

## Score Results

| Review | Score |
|---|---|
| **Global Score** | **45/100** |
| Assure KYC | Not completed |
| Audit Score | 45/100 |

The Following Score System Has been Added to this page to help understand the value of the audit, the maximum score is 100, however to attain that value the project must pass and provide all the data needed for the assessment. Our Passing Score has been changed to 84 Points for a higher standard, if a project does not attain 85% is an automatic failure. Read our notes and final assessment below. The Global Score is a combination of the evaluations obtained between having or not having KYC and the type of contract audited together with its manual audit.

## Audit FAIL

Following our comprehensive security audit of the token contract for the **GG PROTOCOL** project, the project did not fulfill the necessary criteria required to pass the security audit.

# Disclaimer

Assure Defi has conducted an independent security assessment to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the reviewed code for the scope of this assessment. This report does not constitute agreement, acceptance, or advocating for the Project, and users relying on this report should not consider this as having any merit for financial Token in any shape, form, or nature. The contracts audited do not account for any economic developments that the Project in question may pursue, and the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude, and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are entirely free of exploits, bugs, vulnerabilities or deprecation of technologies. All information provided in this report does not constitute financial or investment in Token, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence, regardless of the findings presented. Information is provided ‘as is, and Assure Defi is under no covenant to audit completeness, accuracy, or solidity of the contracts. In no event will Assure Defi or its partners, employees, agents, or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions or actions with regards to the information provided in this audit report. The assessment of Tokens provided by Assure Defi are subject to dependencies and are under continuing development. You agree that your access or use, including but not limited to any Tokens, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies with high levels of technical risk and uncertainty. The assessment reports could include false positives, negatives, and unpredictable results. The Token may access, and depend upon, multiple layers of third parties.